

# Sistema Centralizado de Gestión y Control del Servicio Transmetro (SCG-Transmetro)

## Entregable Fase 5: Arquitectura de Software y Stack Tecnológico

---

**Cliente:** Mancomunidad Gran Ciudad del Sur y Municipalidad de Guatemala

**Firma Consultora:** Consultores Asociados de Sistemas e Ingeniería de Software S.A.  
(CASIS Guatemala)

**Ubicación:** Ciudad de Guatemala, Guatemala

**Versión:** 1.0 — Especificación de Arquitectura

# Índice

---

## 1 Introducción

---

El presente documento define la arquitectura tecnológica que soportará el **Sistema Centralizado de Gestión y Control del Servicio Transmetro (SCG-Transmetro)**. La selección del stack tecnológico obedece a criterios de alta disponibilidad, seguridad nativa, escalabilidad bajo demanda y uso eficiente de los recursos de hardware en el Datacenter Municipal. La solución adopta un enfoque *On-Premise* contenerizado, permitiendo un despliegue ágil, portabilidad entre entornos y aislamiento de procesos críticos.

## 2 Arquitectura Tecnológica del Sistema

---

La plataforma se ha orquestado integrando cuatro pilares fundamentales de software de código abierto y grado empresarial:

### 2.1 Sistema Operativo Base: Fedora Server

La infraestructura física operará sobre **Linux Fedora Server**. Esta distribución se selecciona por:

- **Kernel Moderno y Optimizado:** Proporciona las últimas mejoras en rendimiento de red y gestión de memoria, cruciales para el alto tráfico de telemetría proveniente de las 1,000 estaciones.
- **Seguridad Reforzada:** Integración profunda con *SELinux* activado en modo *enforcing*, previniendo escalada de privilegios y aislando a nivel de kernel las intrusiones externas.

### 2.2 Motor de Contenedores: Podman

En reemplazo de la arquitectura tradicional de Docker, se implementa **Podman** para la orquestación y despliegue de los microservicios:

- **Arquitectura Daemonless:** Al no requerir un demonio central ejecutándose en segundo plano con privilegios de root, reduce drásticamente la superficie de ataque.
- **Gestión de Pods Nativa:** Permite agrupar contenedores relacionados (ej. la aplicación web y su caché temporal) compartiendo espacios de red y almacenamiento de forma análoga a Kubernetes, facilitando una futura migración a K3s/K8s.

### 2.3 Backend y Capa de Presentación: Python y Streamlit

La lógica de negocio, reglas de despacho, y el Centro de Control Visual están desarrollados en **Python** utilizando **Streamlit**:

- **Agilidad de Desarrollo:** Permite la creación inmediata de cuadros de mando (dashboards) interactivos, renderizando componentes HTML y métricas en tiempo real directamente desde scripts en Python.
- **Visualización en Tiempo Real:** Ideal para presentar a los despachadores y a la Mancomunidad los mapas de operación, alertas de sobrecupo ( $\geq 150\%$ ) y ubicación de la flota sin necesidad de un complejo desarrollo frontend/backend por separado.

### 2.4 Capa de Persistencia: MariaDB

El almacenamiento transaccional y relacional es delegado a un clúster **MariaDB**:

- **Rendimiento Transaccional:** Optimizado para un alto volumen de operaciones CRUD y consultas (telemetría de aforo, llegadas y salidas de unidades).
- **Integridad de Datos:** Asegura el cumplimiento estricto de las reglas de negocio normalizadas en 3NF para expedientes de pilotos, inventario de parqueos y trazabilidad de buses operando en las rutas intermunicipales.

### 3 Diagrama de la Arquitectura del Sistema

A continuación, se detalla visualmente la topología de la solución. El diagrama ilustra el flujo de los datos desde las terminales *Edge* en las estaciones, conectadas vía enlaces municipales y VPN, hacia el Centro de Datos orquestado por Podman sobre Fedora Server, culminando en la presentación mediante Streamlit y el resguardo de la información en MariaDB.

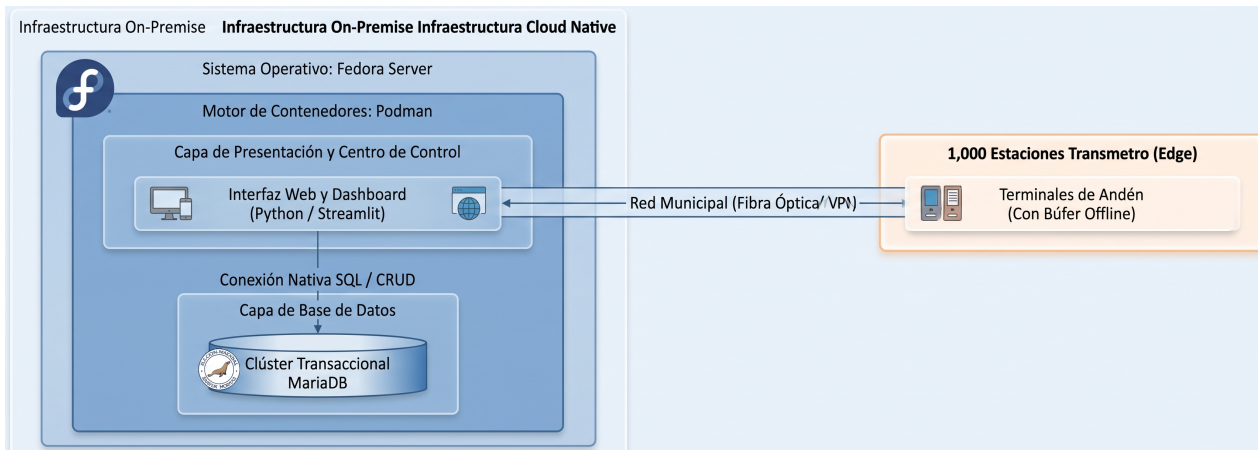


Figura 1: Topología y Arquitectura del Sistema SCG-Transmetro